Operating Systems

ASSIGNMENT 1

Sanjay Dharshan P.N.

192511210

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/192511210simats-sanjay/CSA0411-192511210-assignment1

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking.**

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment 3 Marks

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server
- A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack)
- How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file

- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
    //   stop when read returns 0 (end of file) or -1; write exactly the n bytes that
    //   were read

    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");      /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                          /* empty the buffer before fork() */
    // TODO 1: Create a child with fork(). In the child, replace the program with
    //   execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //   fails. In the parent, wait for the child with waitpid() and print "job 1 exit
    //   status N" using WEXITSTATUS



    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);                  /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner
- The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare 4 Marks

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements**.

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

TASK 1 – EVALUATE THE REQUIREMENTS OF EACH COMPUTING ENVIRONMENT

1.1 Ranked Requirements of Each Environment

Environment	Ranked requirements	Justification
Patient monitors	1. Real-time response 2. Reliability 3. Safety 4. Security 5. Low memory	Must generate alarms within 10 milliseconds.
Nurses' tablets	1. Battery efficiency 2. Security 3. Reliability 4. Responsiveness 5. Portability	Must operate throughout a nursing shift.
Doctors' workstations	1. Security 2. Responsiveness 3. Multitasking 4. Compatibility 5. Reliability	Must support medical applications and patient records.
Database server	1. Availability 2. Data integrity 3. Security 4. Reliability 5. Throughput	Patient records must remain accessible.
MRI server	1. Processing throughput 2. Parallel processing 3. Reliability 4. Memory 5. Security	Must process large MRI images using multiple processors.
Partner network	1. Security 2. Reliability 3. Communication 4. Availability 5. Scalability	Must securely share records between hospitals.

1.2 Explanation of Requirements

1. Bedside Patient Monitors

- Response time must be below 10 milliseconds.
- Reliability is essential because monitoring operates continuously.
- Memory usage should be low.
- Security must prevent unauthorized modification of readings.
- A missed deadline could endanger a patient's life.

2. Nurses' Handheld Tablets

- Battery consumption must be minimized.
- The OS should support wireless communication.
- Encryption and authentication are essential.
- The interface should be easy to use.
- Background processes should be controlled.

3. Doctors' Workstations

- Must support multitasking.
- Should provide a graphical user interface.
- Must support secure access to patient records.
- Should provide sufficient memory and processing performance.

4. Database Server

- Must provide continuous availability.
- Transactions must preserve data integrity.
- Backup and recovery facilities are essential.
- Access permissions must prevent unauthorized access.
- Updates should not interrupt services.

5. MRI Imaging Server

- Requires high processing throughput.
- Must support multiple processors.
- Large memory capacity is required.
- Parallel processing improves performance.

6. Partner Hospital Network

- Requires secure communication.
- Data must be encrypted during transmission.
- Authentication is required between hospitals.
- Distributed access should remain reliable.

1.3 Conflicting Requirements

Conflicting requirements	**Explanation**
Fast response vs. high throughput	Real-time monitors require predictable CPU time while servers serve multiple users.
Security vs. performance	Encryption requires additional processing.
Battery efficiency vs. performance	High processing activity increases energy consumption.
Reliability vs. maintenance	Server updates may interrupt services without fail-over.
Memory vs. functionality	More OS services require additional memory.

1.4 Patient-Safety-Critical Requirements

1. Guaranteed alarm response within 10 milliseconds.
2. Continuous monitoring without system crashes.
3. Accurate storage of patient records.
4. Secure access to confidential information.
5. Reliable backup and recovery.
6. Immediate notification of abnormal readings.

1.5 Application of OS History

OS development	Hospital application
Batch operating systems	Scheduled backups and reports.
Time-sharing systems	Multiple doctors accessing databases.
Personal computer OS	Doctors' workstations.
Mobile operating systems	Nurses' tablets.
Real-time operating systems	Patient monitors.
Distributed systems	Sharing records between hospitals.

1.6 Alternative Concept and Complexity Analysis

If the hospital uses one general-purpose OS everywhere, administration becomes simpler but performance and safety may be affected.

A general-purpose OS may not guarantee a maximum scheduling delay. A hard real-time OS provides predictable scheduling to meet critical deadlines.

Alarm response:

\[T_{response}=T_{interrupt}+T_{scheduling}+T_{processing} \]

The requirement is:

\[T_{response}\leq10ms \]

Memory complexity:

- General-purpose OS requires more memory for background services and graphical interfaces.
- Embedded RTOS uses less memory because unnecessary services can be removed.

Administration effort:

- One OS reduces maintenance complexity.

- Different OS platforms require additional administration and testing.
- Specialized systems provide better safety and efficiency.

Evaluation: A single general-purpose OS is rejected because predictable alarm response is more important than administrative simplicity.

TASK 2 – JUDGE THE TYPE OF OPERATING SYSTEM FOR EACH ENVIRONMENT

2.1 Selection of Operating-System Types

Environment	Recommended OS type	Reason
Patient monitors	Hard real-time OS	Guarantees critical deadlines.
Nurses' tablets	Handheld/mobile OS	Battery management and wireless support.
Doctors' workstations	Desktop OS	GUI and multitasking.
Database server	Clustered server OS	Availability and fail-over.
MRI server	Multiprocessor OS	Parallel processing.
Partner network	Distributed architecture	Sharing records across hospitals.

2.2 Comparison of Operating-System Types

OS type	Advantages	Limitations
Mainframe	High transaction capacity	Expensive
Desktop	User-friendly	Not suitable for hard real-time deadlines
Multiprocessor	High processing capacity	Synchronization complexity
Distributed	Resource sharing	Network dependency
Clustered	Fail-over and availability	Additional configuration
Hard real-time	Guaranteed deadlines	Less flexibility
Soft real-time	Prioritizes important tasks	Deadlines may be missed
Handheld	Battery management	Limited resources

2.3 Symmetric vs. Asymmetric Multiprocessing

Symmetric Multiprocessing (SMP):

- All processors share the same memory.
- Each processor can execute OS and application tasks.
- Workloads can be distributed among processors.
- Suitable for MRI image processing.

Asymmetric Multiprocessing (AMP):

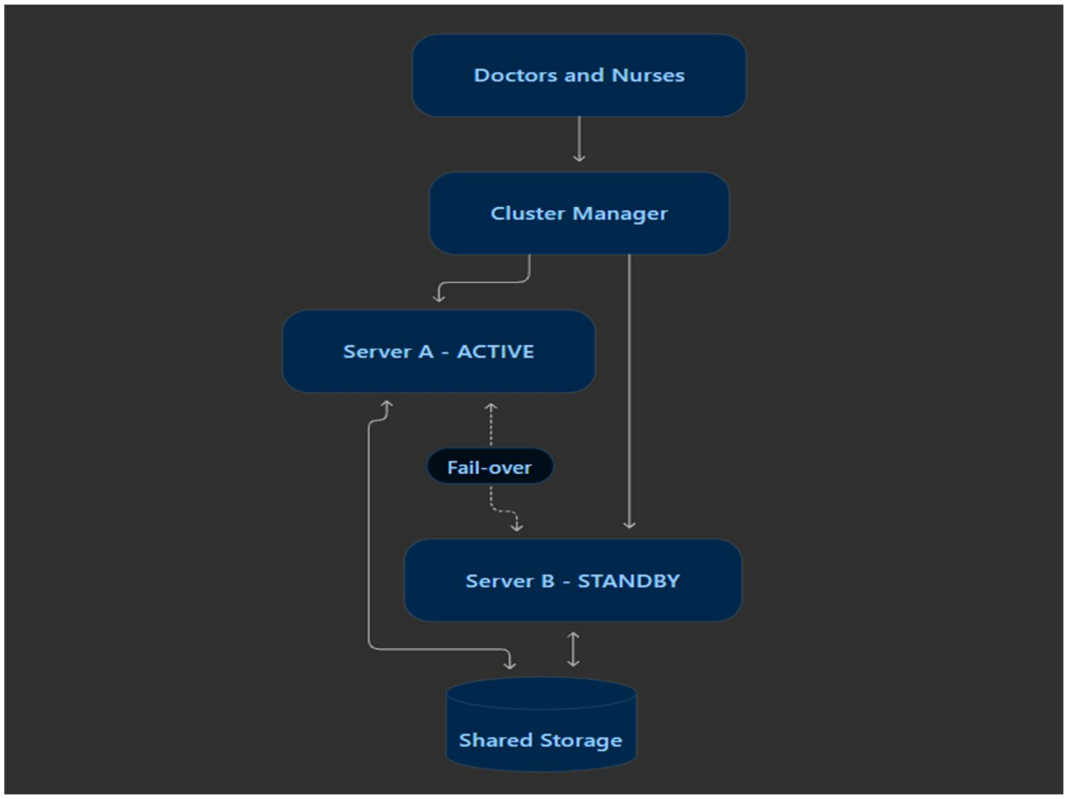
- One processor controls the system.
- Other processors perform assigned tasks.
- More specialized task allocation is possible.
- Useful in dedicated embedded systems.

Decision: SMP is preferred for the MRI server because MRI processing involves large parallel workloads.

2.4 Database Server Clustering

A clustered system uses multiple servers to provide continuous service.

Database cluster architecture:



If Server A fails, Server B takes over the service.

2.5 Alternative Concept and Complexity Analysis

If one large multiprocessor machine is used instead of a cluster:

Factor	**Single multiprocessor**	**Cluster**
Availability	Single point of failure	Fail-over possible
Cost	Expensive hardware	Multiple smaller servers
Throughput	Depends on processor scaling	Can scale across machines
Memory	Limited by one machine	Distributed across nodes
Maintenance	May interrupt service	Fail-over supports maintenance

Amdahl's law:

$$S(N)=\frac{1}{(1-P)+P/N}$$

Where:

- N = number of processors.
- P = parallelizable workload.
- S(N) = theoretical speedup.

Evaluation: A cluster is preferred because patient database availability is more important than relying on one powerful machine.

TASK 3 – EVALUATE OPERATING-SYSTEM STRUCTURES

3.1 Comparison of OS Structures

Structure	**Reliability**	**Performance**	**Updating**
Monolithic	Faulty kernel component may crash system	Fast	Difficult to isolate components
Layered	Better organization	Layer transitions add overhead	Easier maintenance
Microkernel	Strong service isolation	Message passing adds overhead	Easier service updates
Modular	Flexible components	Good performance	Modules can be updated separately

Hybrid	Combines different structures	Balanced performance	Flexible

3.2 Operating-System Examples

OS	**Structure**
Linux	Monolithic kernel with loadable modules
Windows	Hybrid kernel
Android	Linux kernel with Android services
FreeRTOS	Compact embedded RTOS
QNX	Microkernel architecture

3.3 Structure Suitable for Patient Monitors

Recommended: Microkernel structure

Reasons:

1. Essential scheduling functions remain in the kernel.
2. Drivers and services can run separately.
3. A faulty driver is less likely to crash the entire OS.
4. Services can be isolated.
5. The system is easier to maintain and test.

3.4 Structure Suitable for Database Servers

Recommended: Modular kernel structure

Reasons:

- Supports high performance.
- Supports loadable drivers and modules.
- Allows necessary components to be added.
- Provides flexibility for storage and network operations.
- Linux is a suitable example.

3.5 Alternative Concept and Complexity Analysis

If bedside monitors use a monolithic kernel instead of a microkernel:

Factor	**Monolithic**	**Microkernel**
Service request	Direct kernel call	Message passing may be required
Performance	Lower communication overhead	Additional IPC overhead
Kernel memory	More services in kernel	Smaller kernel core
Driver failure	Can crash the system	Better isolation
Reliability	Lower fault isolation	Better fault isolation

Evaluation: Although microkernels introduce communication overhead, fault isolation is more important for life-critical monitors.

TASK 4 – EVALUATE THE SERVICES AN OPERATING SYSTEM MUST PROVIDE

4.1 OS Service Comparison

OS service	**Monitors**	**Tablets**	**Workstations**	**Database**	**MRI server**
User interface	Minimal	Touch UI	GUI	Admin interface	Admin interface
Program execution	Essential	Essential	Essential	Essential	Essential
I/O operations	Essential	Essential	Essential	Essential	Essential
File system	Minimal logging	Essential	Essential	Advanced	Advanced
Communication	Sensor communication	Wi-Fi	Network	Network	Network
Error detection	Critical	Essential	Essential	Critical	Critical
Resource allocation	Real-time	Power-aware	Multitasking	High-throughput	Multiprocessor
Accounting	Minimal	Optional	Optional	Essential	Useful
Protection	Essential	Essential	Essential	Critical	Critical

Security	Critical	Critical	Critical	Critical	Critical

4.2 Services That Can Be Removed From Bedside Monitors

The following services can be reduced or excluded:

- Full graphical desktop environment.
- Unnecessary background applications.
- Printing services.
- Unused network protocols.
- Unnecessary multimedia services.
- Unneeded drivers.

The following must remain:

- Real-time scheduling.
- Sensor input and output.
- Alarm generation.
- Error detection.
- Secure communication.
- Essential logging.
- Memory protection.

4.3 Essential Database Services

1. Process management.
2. Memory management.
3. File management.
4. Network communication.
5. User authentication.
6. Access control.
7. Error detection.
8. Backup and recovery.
9. Resource allocation.
10. Auditing and accounting.

4.4 Protection of Patient Records

The OS protects records using:

- User authentication.
- Role-based access control.
- File permissions.
- Process isolation.
- Memory protection.
- Encryption.
- Audit logs.

For example, a nurse should only access records required for assigned patients.

4.5 Alternative Concept and Complexity Analysis

If tablets use a full desktop OS instead of a reduced handheld OS:

Factor	**Full desktop OS**	**Handheld OS**
Memory	Higher	Lower
Storage	More system files	Fewer system files
Startup	More services	Usually faster
Battery	More background consumption	Better power optimization
Security	More components to maintain	Smaller attack surface when configured correctly

Evaluation: A reduced handheld OS is preferable because battery life, portability and efficient resource usage are essential.

TASK 5 – EXAMINE SYSTEM CALLS (4 MARKS)

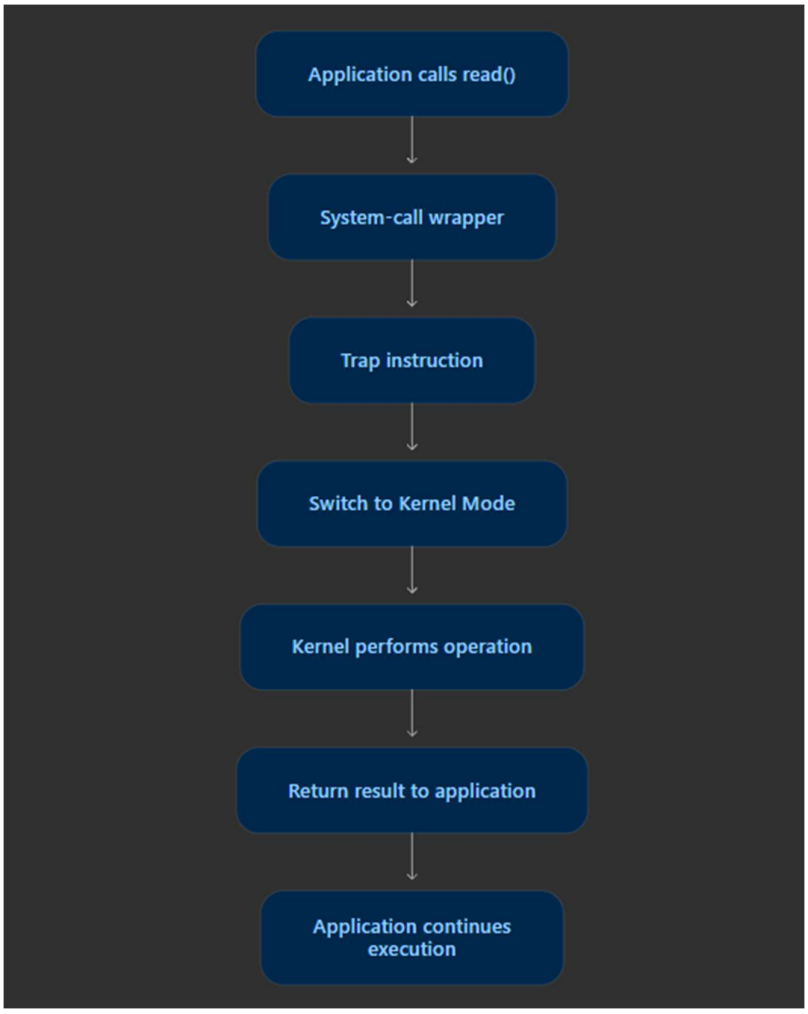
5.1 Categories of System Calls

Category	**Purpose**	**Examples**
Process control	Create and manage processes	fork(), exec(), wait(), exit()
File management	Create, open, read and write files	open(), read(), write(), close()
Device management	Control devices	ioctl(), read(), write()
Information maintenance	Obtain system information	getpid(), time(), uname()
Communication	Exchange information	pipe(), socket(), send(), recv()
Protection	Manage permissions	chmod(), chown(), access()

5.2 Step-by-Step Working of read()

1. The application calls the read() library interface.
2. The system-call wrapper prepares parameters.
3. A trap instruction transfers control to the kernel.
4. The processor switches from user mode to kernel mode.
5. The kernel validates the file descriptor and operation.
6. The kernel obtains the requested data.
7. The result or error code is prepared.

System-call execution:



5.3 Parameter-Passing Methods

1. Registers: Parameters are placed in CPU registers.
2. Memory block: Parameters are stored in memory and the address is passed.
3. Stack: Parameters are pushed onto the stack.

5.4 Using strace

strace displays system calls made by a Linux program.

Commands:

strace -o trace.txt ./copytest

strace -e trace=read,write ./copytest

strace -c ./copytest

Example output format:

% time seconds usecs/call calls errors syscall

------ ------- ---------- ----- ------ -------

read ...

write ...

openat ...

close ...

The exact output depends on the Linux environment and program execution.

5.5 Alternative Concept and Complexity Analysis

Using fopen(), fread() and fwrite() instead of direct system calls introduces library buffering.

The approximate number of system calls is:

\[N\approx\left\lceil\frac{n}{b}\right\rceil \]

Where:

- n = file size.
- b = buffer size.

Library buffering reduces expensive transitions between user mode and kernel mode but requires additional buffer memory.

TASK 6 – EVALUATE THE COST OF SYSTEM CALLS

6.1 Why System Calls Are Expensive

An ordinary function call generally stays in user mode. A system call requires a transition into kernel mode.

Additional operations include:

- Saving processor state.
- Switching execution modes.
- Validating parameters.
- Executing kernel operations.
- Returning to user mode.
- Restoring processor state.

Therefore, excessive system calls can reduce performance.

6.2 Completed C Program – TODO 1

Filename: copytest.c

```
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

long copy_file(const char *src, const char *dst,
               size_t bufsize)
{
   char buf[8192];

   if (bufsize == 0 || bufsize > sizeof(buf))
      return -1;

   int in = open(src, O_RDONLY);
   if (in < 0) return -1;
```

```
int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);

if (out < 0) {
   close(in);
   return -1;
}

long calls = 0;
ssize_t n;

while (1) {
   do {
      n = read(in, buf, bufsize);
      calls++;
   } while (n < 0 && errno == EINTR);

   if (n == 0)
      break;

   if (n < 0) {
      calls = -1;
      break;
   }

   ssize_t written = 0;

   while (written < n) {
      ssize_t w = write(
         out,
         buf + written,
```

```
            n - written
        );

        if (w < 0 && errno == EINTR)
            continue;

        if (w <= 0) {
            calls = -1;
            goto finish;
        }

        written += w;
      }
   }

finish:
   close(in);
   close(out);

   return calls;
}

int main(void)
{
   FILE *f = fopen("records.dat", "w");

   if (!f) return 1;

   for (int i = 0; i < 10000; i++)
      fputc('A' + i % 26, f);
```

```
    fclose(f);

    size_t sizes[] = {1, 512, 4096};

    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n",
               sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));

    unlink("records.dat");
    unlink("copy.dat");

    return 0;
}
```

6.3 Compilation and Execution

```
gcc -Wall -Wextra copytest.c -o copytest
./copytest
```

Expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

6.4 Comparison of Buffer Sizes

Buffer size	**Read calls**	**Explanation**
1 byte	10001	10000 data reads and one EOF read
512 bytes	21	20 data reads and one EOF read
4096 bytes	4	3 data reads and one EOF read

The number of calls is:

$$ N=\left\lceil\frac{10000}{b}\right\rceil+1 $$

6.5 Best Buffer Size for the Hospital Backup Program

Recommended buffer size: 4096 bytes or larger, subject to benchmarking.

Reasons:

- Reduces system calls.
- Reduces kernel-mode transitions.
- Improves throughput.
- Requires relatively little memory.
- Matches a common memory-page size.

For large backups, 64 KB and 256 KB should also be tested.

6.6 Alternative Concept and Complexity Analysis – mmap()

Memory-mapped files allow a file to be accessed through virtual memory.

Factor	**read()/write()**	**mmap()**
System calls	Approximately n/b	A few setup and cleanup calls
Data copying	Copies into user buffers	Avoids explicit read into user buffer
Memory	Buffer memory	Mapped virtual address range
Large files	Processed in chunks	Can map selected regions
Performance	Depends on buffer size	Depends on page faults and storage

Evaluation: mmap() is useful for random access. Buffered read() and write() remain suitable for sequential backup copying.

TASK 7 – EVALUATE PROCESS CONTROL SYSTEM CALLS

7.1 Functions of Process-Control System Calls

Function	Purpose
fork()	Creates a child process.
exec()	Replaces the current process image.
wait()	Waits for a child process.
waitpid()	Waits for a particular child.
exit()	Terminates a process.

7.2 How a Shell Executes a Command

1. The shell reads the command.
2. It creates a child process using fork().
3. The child calls exec().
4. The new program replaces the child process image.
5. The parent waits using waitpid().
6. The child finishes and returns an exit status.
7. The parent obtains and displays the exit status.

7.3 Completed C Program – TODO 1

Filename: process.c

```
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
   pid_t pid;
   int status;
```

```
printf("shell: running job 1\n");
fflush(stdout);

pid = fork();

if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    execlp(
        "expr",
        "expr",
        "6",
        "*",
        "7",
        (char *)NULL
    );

    perror("execlp");
    _exit(127);
}

if (waitpid(pid, &status, 0) < 0) {
    perror("waitpid");
    return 1;
}

if (WIFEXITED(status))
```

```
        printf("job 1 exit status %d\n",
                WEXITSTATUS(status));

    printf("shell: running job 2\n");
    fflush(stdout);

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0)
        _exit(3);

    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return 1;
    }

    if (WIFEXITED(status))
        printf("job 2 exit status %d\n",
                WEXITSTATUS(status));

    return 0;
}
```

7.4 Compilation and Execution

```
gcc -Wall -Wextra process.c -o process
./process
```

Expected output:

shell: running job 1

42

job 1 exit status 0

shell: running job 2

job 2 exit status 3

7.5 Zombie and Orphan Processes

Zombie process:

A zombie is a terminated child process whose parent has not collected its exit status.

The parent removes this entry using wait() or waitpid().

Orphan process:

An orphan is a process whose parent terminates before it.

The operating system reassigns it to an appropriate system process.

7.6 Why stdout Is Flushed Before fork()

Standard output can be buffered.

If output remains in the buffer when fork() occurs, the child receives a copy of that buffer.

When both processes call exit(), the same buffered message may be written twice.

Flushing before fork() prevents duplication of previously buffered output.

When the second fflush() is removed and stdout is redirected to a file, the child inherits the pending message. If the child calls exit(3), it flushes its copy. The parent later flushes its copy, causing the message to appear twice.

_exit(3) does not flush standard I/O buffers.

7.7 Alternative Concept and Complexity Analysis

Factor	fork() + exec()	posix_spawn()	Threads
Startup	Process creation and replacement	Often efficient for simple launches	Generally lower overhead
Memory	Copy-on-write	Avoids some fork overhead	Shared address space
Isolation	Strong	Strong	Limited
Failure handling	Separate process status	Separate process status	Shared resources

Evaluation: posix_spawn() is suitable for launching independent commands. Processes are preferred over threads for executing independent or untrusted hospital applications.

TASK 8 – EVALUATE CPU STATES AND PROTECTION

8.1 User Mode and Kernel Mode

User mode:

- Used by ordinary applications.
- Cannot execute privileged instructions.
- Cannot directly access protected kernel memory.
- Must request OS services through system calls.

Kernel mode:

- Used by the operating-system kernel.
- Can execute privileged instructions.
- Can manage memory and hardware.
- Controls device access and interrupts.

8.2 Mode Bit

The processor uses a mode indicator to distinguish execution privileges.

Mode	**Access**
User mode	Restricted
Kernel mode	Privileged

8.3 Interrupts, Traps and Timer Interrupts

Interrupts are hardware signals that require processor attention.

Traps are synchronous events generated by executing instructions.

Timer interrupts allow the OS to regain CPU control and perform scheduling.

8.4 What Happens If Programs Can Disable Interrupts?

- Sensor events may not be handled immediately.
- Alarm processing could be delayed.
- Other tasks may not receive CPU time.
- The operating system may lose control.
- Patient safety could be affected.

Therefore, disabling interrupts must be privileged.

8.5 Memory Protection

The OS and processor use:

- Virtual memory.
- Page tables.
- Memory protection bits.
- Process address-space isolation.
- Privileged execution modes.

A program cannot directly read another process's protected memory unless an authorized sharing mechanism is provided.

8.6 Alternative Concept and Complexity Analysis – Polling vs. Interrupts

Factor	**Polling**	**Interrupt-driven**
Detection	Checks repeatedly	Hardware signals an event
Response delay	Depends on polling interval	Depends on interrupt latency
CPU usage	Can waste CPU cycles	CPU can perform other work
Power	May consume more energy	Usually more efficient while idle
Predictability	Depends on polling frequency	Depends on interrupt handling

If polling interval is P:

$$T_{maximum\ detection\ delay}\approx P$$

For example, with a 5 ms polling interval, an abnormal reading could wait almost 5 ms before being noticed.

With interrupt-driven input:

$$T_{response}=T_{interrupt}+T_{processing}+T_{alarm}$$

Evaluation: Interrupt-driven input combined with a hard real-time scheduler is preferred for patient monitors.

TASK 9 – EVALUATE MEMORY HIERARCHY AND I/O SYSTEMS

9.1 Memory Hierarchy

Memory level	Typical access time	Relative capacity
CPU registers	Less than 1 ns	Very small
CPU cache	Approximately 1–20 ns	KB to MB
Main memory	Approximately 50–150 ns	GB
SSD	Approximately 50–200 µs	Hundreds of GB to TB
Hard disk	Approximately 5–15 ms	TB
Network storage	Variable, often ms or more	Large and scalable

These are approximate illustrative values.

9.2 Recommended Data Placement

Data	Recommended storage	Reason
MRI images	SSD and network storage	Fast processing and large capacity
Patient records	SSD with replicated backup	Fast lookup and data protection
Monitor readings	RAM and persistent storage	Low-latency processing and historical logs
Historical records	Network and archival storage	Long-term retention
Frequently accessed records	RAM cache	Faster access

9.3 Caching

Caching keeps frequently accessed information in faster memory.

Advantages:

- Faster record lookup.
- Reduced disk operations.
- Lower response time.
- Improved throughput.

Cached records must be updated correctly to prevent stale patient information.

9.4 I/O Methods

I/O method	Description	Suitable application
Programmed I/O	CPU repeatedly checks and transfers data	Simple device operations
Interrupt-driven I/O	Device interrupts CPU when service is needed	Patient-monitor sensors
DMA	Device transfers data directly to memory	MRI image transfer

9.5 Recommended MRI I/O Method

Direct Memory Access (DMA) is recommended because:

1. MRI images contain large amounts of data.
2. DMA transfers data between device and memory.
3. The CPU does not handle every individual word.
4. CPU time becomes available for image processing.
5. Interrupts can be used when a transfer completes.

9.6 Alternative Concept and Complexity Analysis

Suppose the MRI scanner transfers a 500 MB image using one interrupt per 4-byte word.

Assuming decimal 500 MB:

\[N=\frac{500,000,000}{4} \]

\[N=125,000,000\ interrupts \]

If binary 500 MiB is used:

\[N=131,072,000\ interrupts \]

At an illustrative 1–5 microseconds per interrupt:

\[125,000,000\times1\mu s=125s \]

\[125,000,000\times5\mu s=625s \]

This is an illustrative calculation, not a measured benchmark.

DMA transfers large blocks and requires far fewer interrupts.

Evaluation: DMA is substantially more efficient for large MRI transfers.

TASK 10 – MEASURE AND COMPARE

10.1 Measurement Environment

Component	Environment
Operating system	Linux
Architecture	x86-64
Compiler	GCC
Copy program	C
Process benchmark	fork() and waitpid()
Thread benchmark	pthread_create() and pthread_join()
System-call tracing	strace unavailable

10.2 System-Call Measurements

Buffer size Read calls

1 byte 10001

512 bytes 21

4096 bytes 4

These measurements support the conclusion that increasing buffer size reduces system calls.

10.3 strace Commands

strace -c ./copytest

strace -c -e trace=read,write ./copytest

10.4 User Mode and Kernel Mode Timing

/usr/bin/time -v ./copytest

This displays:

- User CPU time.
- System CPU time.
- Elapsed time.
- Maximum resident memory.
- Page faults.
- Context switches.

10.5 Process and Thread Creation Measurements

Filename: benchmark.c

```
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <pthread.h>
#include <time.h>

static double get_time(void)
{
    struct timespec t;

    clock_gettime(CLOCK_MONOTONIC, &t);

    return t.tv_sec + t.tv_nsec / 1000000000.0;
}

void *worker(void *arg)
{
    (void)arg;
    return NULL;
}

int main(void)
{
    double start, end;

    start = get_time();
```

```
for (int i = 0; i < 1000; i++) {

    pid_t pid = fork();

    if (pid == 0)
        _exit(0);

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    int status;
    waitpid(pid, &status, 0);
}

end = get_time();

printf("1000 processes: %.6f seconds\n",
       end - start);

start = get_time();

for (int i = 0; i < 1000; i++) {

    pthread_t thread;

    if (pthread_create(
        &thread, NULL, worker, NULL) != 0)
        return 1;
```

```
        pthread_join(thread, NULL);
    }

    end = get_time();

    printf("1000 threads: %.6f seconds\n",
           end - start);

    return 0;
}
```

Compile:

```
gcc -O2 -pthread benchmark.c -o benchmark
```

Run:

```
./benchmark
```

10.6 Previously Observed Measurement Results

Operation	Number	Measured time
Processes	1000	0.171853 seconds
Threads	1000	0.036290 seconds

These are single-run measurements from the available execution environment.

10.7 Evaluation of Results

Threads are generally faster to create because:

- They share the same process address space.
- They share code and data.
- They require less memory-management work.
- They avoid creating an independent process address space.

Processes require more resources because:

- Each process has its own virtual address space.
- Process creation involves additional kernel bookkeeping.
- Copy-on-write reduces copying but does not eliminate process-management overhead.

10.8 Alternative Concept and Complexity Analysis

Factor	**Processes**	**Threads**
Creation time	Generally higher	Generally lower
Memory	Separate address spaces	Shared address space
Communication	IPC required	Shared memory available
Isolation	Stronger	Weaker
Failure	Independent process failure	Can affect entire process

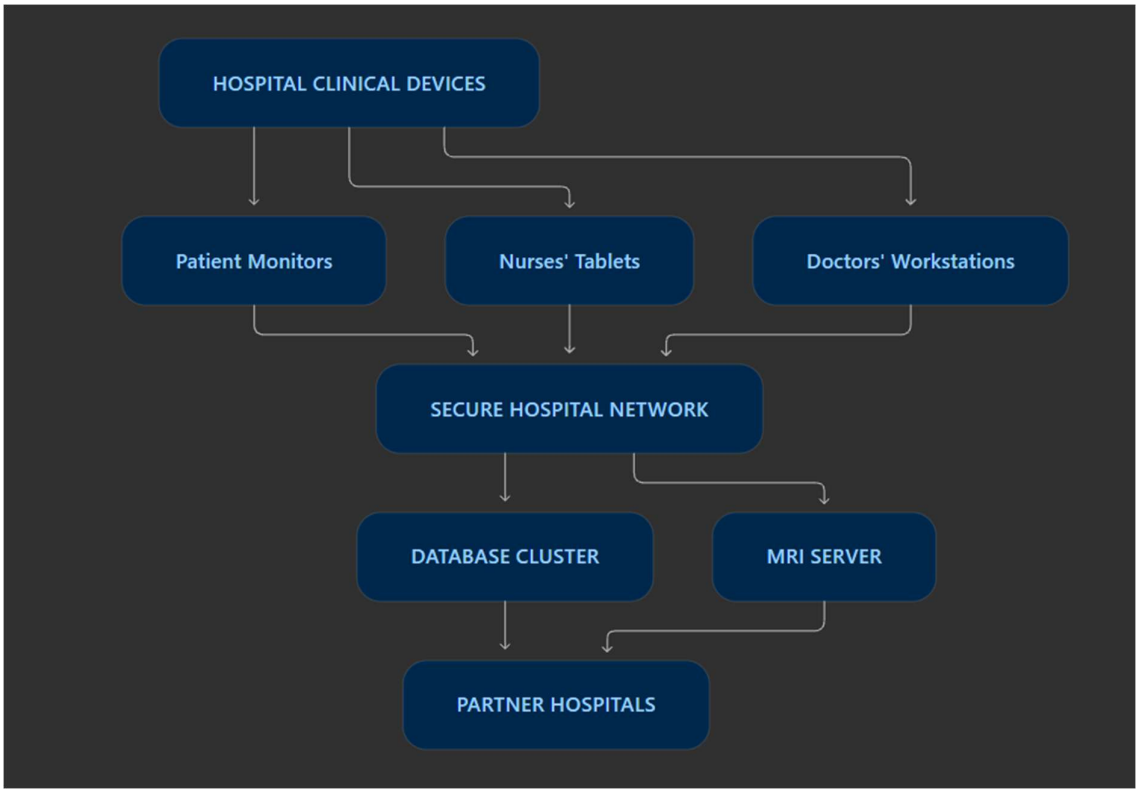
Evaluation: Threads are suitable for parallel image processing. Independent processes are preferable for security-sensitive applications.

TASK 11 – RECOMMEND AND DEFEND THE OPERATING-SYSTEM PLAN

11.1 Complete OS Recommendation

Environment	**OS type**	**OS structure**	**Main services**
Patient monitors	Hard real-time OS	Microkernel preferred	Scheduling, sensor I/O, alarms
Nurses' tablets	Handheld OS	Mobile/hybrid	Wireless, power management, encryption
Doctors' workstations	Desktop OS	Hybrid/modular	GUI, multitasking, authentication
Database server	Clustered server OS	Modular	Storage, networking, backup
MRI server	Multiprocessor OS	Modular	SMP, DMA, memory management
Hospital network	Distributed architecture	Distributed services	Secure communication and replication

11.2 Overall Architecture



11.3 Protection and I/O Methods

Environment	**Protection**	**I/O method**
Monitors	Memory isolation and restricted privileges	Interrupt-driven input
Tablets	Authentication and encryption	Wireless I/O
Workstations	User accounts and permissions	Interrupt-driven I/O
Database	Role-based access and auditing	DMA and buffered I/O
MRI server	Process isolation	DMA
Network	Encryption and authentication	Network communication

11.4 Limitations and Risks

1. Different OS platforms increase maintenance complexity.
2. Hardware failures can interrupt services.
3. Network failures can prevent remote record access.
4. Security vulnerabilities may affect multiple environments.

5. Cluster synchronization requires careful configuration.
6. Real-time performance must be verified.
7. Software updates require testing.

11.5 Scalability

Stage	System size	Architecture
One ward	40 monitors, 10 tablets	Embedded and handheld OS
One hospital	500 monitors, 120 tablets, 80 workstations	Specialized systems and database cluster
Three hospitals	Shared patient records	Distributed architecture

11.7 Final Recommendation

The hospital should use separate specialized operating systems instead of one general-purpose OS everywhere.

The most important reasons are:

- Patient safety requires predictable real-time response.
- Tablets require efficient battery management.
- Database services require high availability.
- MRI workloads require multiprocessing.
- Partner hospitals require secure distributed communication.

The measurements demonstrate that larger buffers reduce system-call overhead and that threads were faster than processes in the tested benchmark.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

Changed Requirement

The hospital adds remote patient-monitoring devices in patients' homes and moves patient records to a cloud service shared with partner hospitals.

A. IMPACT ANALYSIS

The new requirements affect the original design because the system must operate across public networks and external cloud infrastructure.

A.1 Affected OS Types and Services

Component	**Original design**	**New requirement**	**Impact**
Patient monitors	Local hard RTOS	Home monitoring	Embedded RTOS with secure communication
Tablets	Handheld OS	Remote access	Stronger network security
Database	Local cluster	Cloud records	Distributed database services
Partner link	Hospital network	Cloud sharing	Secure distributed architecture
Communication	Local communication	Internet communication	Encryption and authentication
Storage	Local SSD and backups	Cloud storage	Replication and recovery

A.2 Affected Protection Mechanisms

The new design requires:

- End-to-end encryption.
- Secure device authentication.
- Role-based access control.
- Secure communication protocols.
- Multi-factor authentication for administrative access.
- Audit logging.
- Secure software updates.
- Data backup and recovery.
- Network segmentation.

Home devices must not be trusted automatically simply because they belong to registered patients.

B. EVALUATION OF THE ORIGINAL DESIGN

The original design remains suitable for internal hospital infrastructure but needs modifications for home monitoring and cloud-based records.

B.1 Response Time

Local bedside monitors can generate alarms without depending on an internet connection.

However, home-monitoring devices depend on network connectivity to communicate readings.

Network delays can be unpredictable.

Therefore:

- Critical local alarms should be generated on the home device.
- Remote readings should be transmitted to the hospital.
- The hospital should generate additional notifications when abnormal readings arrive.
- Loss of internet connectivity should not disable local monitoring.

B.2 Reliability

Cloud services can provide:

- Geographic redundancy.
- Automated backups.
- Replication.
- Scalable resources.

B.3 Security

The original design needs stronger protection because medical records will be transmitted and stored outside the hospital.

Security must include:

- Encryption during transmission and storage.
- Strong authentication.
- Access permissions.
- Device identity verification.

C. ADAPTATION OR REDESIGN

Decision: MODIFY THE ORIGINAL DESIGN

The original design should be modified rather than completely replaced.

C.1 Revised OS Plan

Environment	**Revised recommendation**
Hospital patient monitors	Retain hard real-time OS
Home monitoring devices	Embedded real-time OS
Nurses' tablets	Retain mobile OS with secure remote access
Doctors' workstations	Retain desktop OS
Hospital database	Retain local cluster for essential operations

C.2 Revised Architecture



C.3 Additional Services

1. Secure device registration.
2. Encrypted data transmission.
3. Cloud storage.
4. Distributed database synchronization.
5. Remote monitoring.
6. Backup and recovery.
7. Identity and access management.

D. EVIDENCE-BASED DEFENCE

The decision to modify the original architecture is based on the following evaluation.

Requirement	**Retained design**	**Required modification**	**Final result**
Local alarm response	Hard RTOS	Maintain local processing	Retain
Home monitoring	Not originally included	Embedded RTOS and secure internet	Modify
Database availability	Local clustering	Cloud replication and local fallback	Modify
Data security	Local authentication	Encryption and stronger identity management	Modify
Partner hospital access	Distributed communication	Cloud-based shared records	Modify
MRI processing	Multiprocessor OS	No major change required	Retain

D.1 Trade-offs

Advantages of modification:

- Supports patients outside the hospital.
- Allows doctors to access shared records.
- Improves scalability.
- Supports centralized backup.
- Provides access across multiple hospitals.

Disadvantages:

- Greater dependence on internet connectivity.
- Increased security risks.
- Cloud service costs.
- Additional administration.
- Potential latency during remote communication.

D.2 Final Defence

The original architecture should not be completely redesigned because its specialized operating systems already satisfy the hospital's internal requirements.

The bedside monitors still need hard real-time operating systems. Tablets still need mobile operating systems. Doctors still need desktop operating systems, while the MRI server still benefits from multiprocessing.

However, home monitoring requires embedded real-time devices with secure communication, and cloud-based records require distributed storage, replication and stronger authentication.

The local hospital infrastructure should continue to support essential clinical functions when the internet or cloud service is unavailable.

Final Decision: MODIFY THE ORIGINAL DESIGN.

This approach preserves the patient-safety benefits of the original architecture while extending it to support remote monitoring, cloud records and three-hospital data sharing.